

Queries used in SQLight for data cleaning and validating

1. Checking that there are the correct amount of rows in my tables

- `SELECT COUNT(*) AS CustomerCount FROM Customers;`
- `SELECT COUNT(*) AS VehicleCount FROM Vehicle;`
- `SELECT COUNT(*) AS PolicyCount FROM Policy;`
- `SELECT COUNT(*) AS ClaimsCount FROM Claims;`

2. Checking that there are no duplicate primary key values

- `SELECT CustomerID, COUNT(*)
FROM Customers
GROUP BY CustomerID
HAVING COUNT(*) > 1;`
- `SELECT VehicleID, COUNT(*)
FROM Vehicle
GROUP BY VehicleID
HAVING COUNT(*) > 1;`
- `SELECT PolicyID, COUNT(*)
FROM Policy
GROUP BY PolicyID
HAVING COUNT(*) > 1;`
- `SELECT ClaimsID, COUNT(*)
FROM Claims
GROUP BY ClaimsID
HAVING COUNT(*) > 1;`

3. Checking for missing or null values

- `SELECT *
FROM Customers
WHERE Name IS NULL
OR Age IS NULL
OR Location IS NULL
OR Gender is NULL`

OR DatePassedDrivingTest is NULL;

- SELECT *
FROM Vehicle
WHERE Brand IS NULL
OR Mileage IS NULL
OR Age is NULL
OR Value IS NULL;
- SELECT *
FROM Policy
WHERE CustomerID IS NULL
OR VehicleID IS NULL
OR Premium IS NULL
OR StartDate is NULL
OR EndDate is NULL
OR LengthOfPolicy is NULL
OR CoverageType is NULL
OR Excess is NULL;
- SELECT *
FROM Claims
WHERE PolicyID IS NULL
OR ClaimDate is NULL
OR ClaimType is NULL
OR ClaimStatus is NULL
OR Amount IS NULL;

4. Checking the maximum and minimum age of the customers and that they do not go above or below the expected age limit

- SELECT MIN(Age) AS MinAge,
MAX(Age) AS MaxAge
FROM Customers;
- SELECT *
FROM Customers
WHERE Age < 17
OR Age > 100;

5. Checking the minimum, maximum and average value of the cars in the vehicle table as well as if there are any cars with zero value or mileage less than zero

- SELECT MIN(Value) AS MinValue,

```
MAX(Value) AS MaxValue,  
AVG(Value) AS AvgValue  
FROM Vehicle;
```

- SELECT *
FROM Vehicle
WHERE Value <= 0
OR Mileage < 0;

6. Confirming if the policies end date is after the policies start date

- SELECT *
FROM Policy
WHERE EndDate <= StartDate;

7. Confirming the policy length matches the actual length of the policy

- SELECT PolicyID,
StartDate,
EndDate,
LengthOfPolicy,
ROUND((julianday(EndDate) - julianday(StartDate))/365.25,2) AS ActualYears
FROM Policy;

8. Checking that every foreign key is linked to a valid primary key

- SELECT c.*
FROM Claims c
LEFT JOIN Policy p
ON c.PolicyID = p.PolicyID
WHERE p.PolicyID IS NULL;
- SELECT p.*
FROM Policy p
LEFT JOIN Customer c
ON p.CustomerID = c.CustomerID
WHERE c.CustomerID IS NULL;
- SELECT p.*
FROM Policy p
LEFT JOIN Vehicle v
ON p.VehicleID = v.VehicleID

WHERE v.VehicleID IS NULL;

9. Confirming there are no negative claims or too many zero value claims/approved claims with zero payout

- ```
SELECT *
FROM Claims
WHERE Amount < 0;
```
- ```
SELECT *  
FROM Claims  
WHERE Amount = 0;
```
- ```
SELECT COUNT(*) AS ApprovedZeroClaims
FROM Claims
WHERE ClaimStatus = 'Approved'
AND Amount = 0;
```

## 10. Confirming there are no claims that have been denied, with positive payouts

- ```
SELECT *  
FROM Claims  
WHERE ClaimStatus = 'Denied'  
AND Amount > 0;
```

11. Checking the coverage type distribution

- ```
SELECT CoverageType,
COUNT(*) AS Policies
FROM Policy
GROUP BY CoverageType;
```

## 12. Checking the claim type distribution

- ```
SELECT ClaimType,  
COUNT(*) AS Claims  
FROM Claims
```

```
GROUP BY ClaimType
ORDER BY Claims DESC;
```

13. Checking the gender distribution

- ```
SELECT Gender,
COUNT(*) AS Customers
FROM Customer
GROUP BY Gender;
```

### 14. Checking the average premium paid per age bands

- ```
SELECT
CASE
WHEN Age < 25 THEN '18-24'
WHEN Age < 40 THEN '25-39'
WHEN Age < 60 THEN '40-59'
ELSE '60+'
END AS AgeBand,
ROUND(AVG(Premium),2) AS AvgPremium
FROM Customer c
JOIN Policy p
ON c.CustomerID = p.CustomerID
GROUP BY AgeBand;
```

15. Checking the claim frequency percentage is similar to the claim frequency used in data generation

- ```
SELECT
ROUND(
(COUNT(DISTINCT Claims.PolicyID) * 100.0)
/ COUNT(DISTINCT Policy.PolicyID),
2) AS ClaimFrequencyPercent
FROM Policy
LEFT JOIN Claims
ON Policy.PolicyID = Claims.PolicyID;
```

### 16. Unrealistic premium checks

- ```
SELECT *
FROM Policy
WHERE Premium < 200
```

OR Premium > 10000;

17. Checking the location distribution

- SELECT Location,
COUNT(*) AS Customers
FROM Customer
GROUP BY Location
ORDER BY Customers DESC
LIMIT 20;

18. Confirming that there are no claims greater than the vehicle value

- SELECT c.ClaimsID,
c.Amount,
v.Value
FROM Claims c
JOIN Policy p
ON c.PolicyID = p.PolicyID
JOIN Vehicle v
ON p.VehicleID = v.VehicleID
WHERE c.Amount > v.Value;